PHP–MySQL Application Development

Database Concepts, MySQL Basics, PHP Connections, and CRUD Operations

Learning Objective

- Explain fundamental database concepts and terminology
- Write basic SQL queries (CREATE, INSERT, SELECT, UPDATE, DELETE)
- Connect PHP applications to MySQL using MySQLi
- Implement CRUD operations in PHP–MySQL applications

What is a Database?

- Recall the basic reason for server-side programming
  - We need to store client data or look up data stored on the server
- **Databases** give us an easy way to issue “commands” to insert, select, organize, and remove data
  - A database is an organized collection of related data.
- Purpose:
  - Store large amounts of data efficiently
  - Allow fast searching and retrieval
  - Support multiple users simultaneously

Why Use Databases?

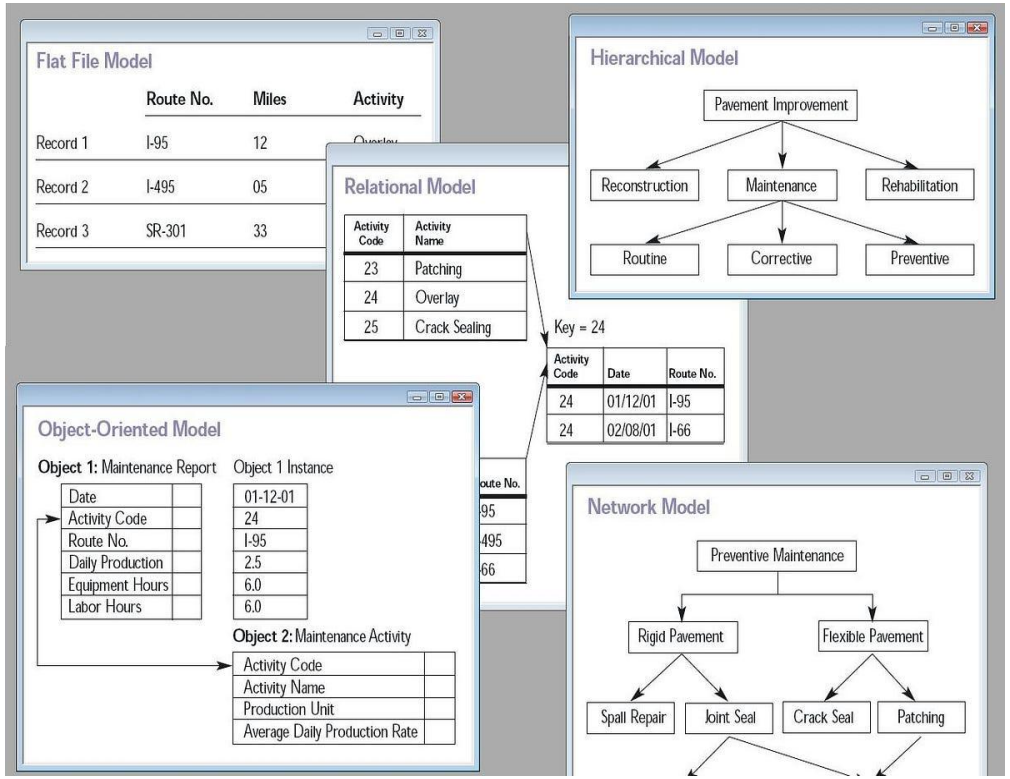
- Databases are essential for modern web applications.
  - Reduces data redundancy
  - Improves data consistency
  - Enhances data security
  - Supports backup and recovery

Recall: Basic Database Terminology

- **Table**: structure that stores data
- **Record (Row):** single entry in a table
- **Field (Column):** attribute of a record
- **Schema**: overall database structure
- **Relational Databases**: organize data into tables

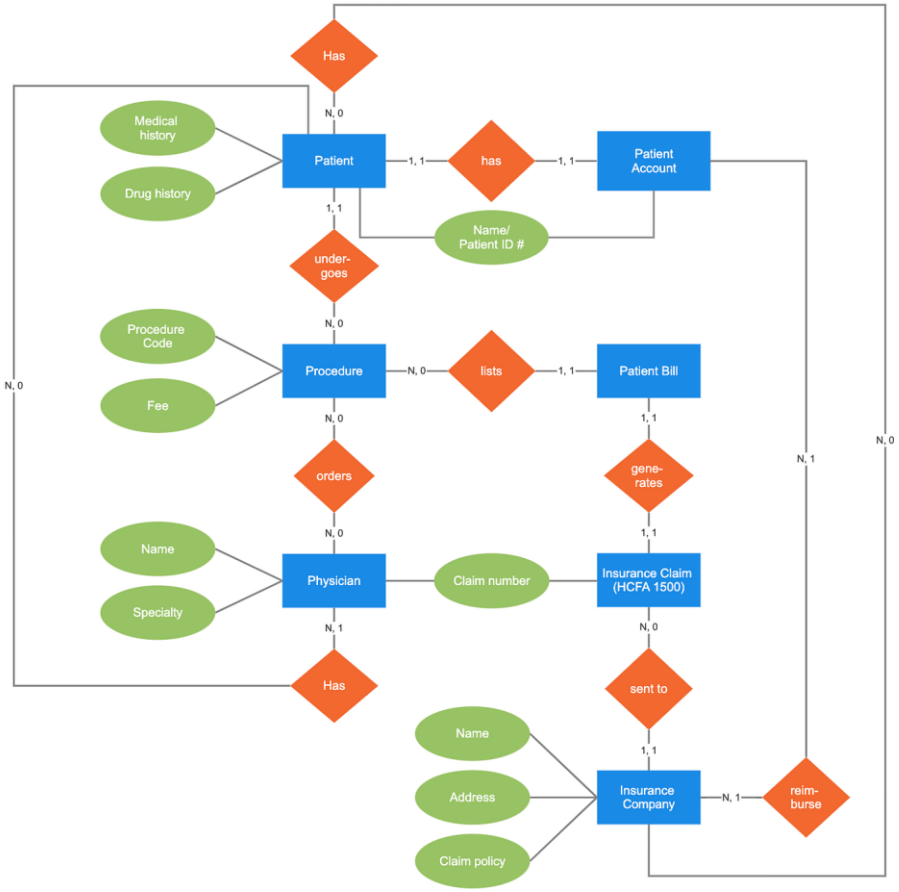
  Characteristics:

  - Tables are related through keys
  - Uses SQL for data manipulation
  - Enforces data integrity rules



Recall: What Is an ERD?

- **Entity Relationship Diagram** (ERD) is a visual representation of a database.
- It shows:
  - **Entities** – represents the real-world object. It becomes a table in database
  - **Attributes** – describe the properties of an entity
  - **Relationships** – describe how entities are connected
- Why ERD is important?
  - Helps plan database structure
  - Prevents data duplication
  - Makes database easier to understand
  - Saves development time



Introduction to MySQL

What is MySQL?

- MySQL is a Relational Database Management System (RDBMS).
- Open-source database relatively easy to setup, easy to use with PHP
- MySQL database server can contain many databases, each of which can contain many tables
- MySQL is ideal for both small and large applications
- MySQL supports standard SQL

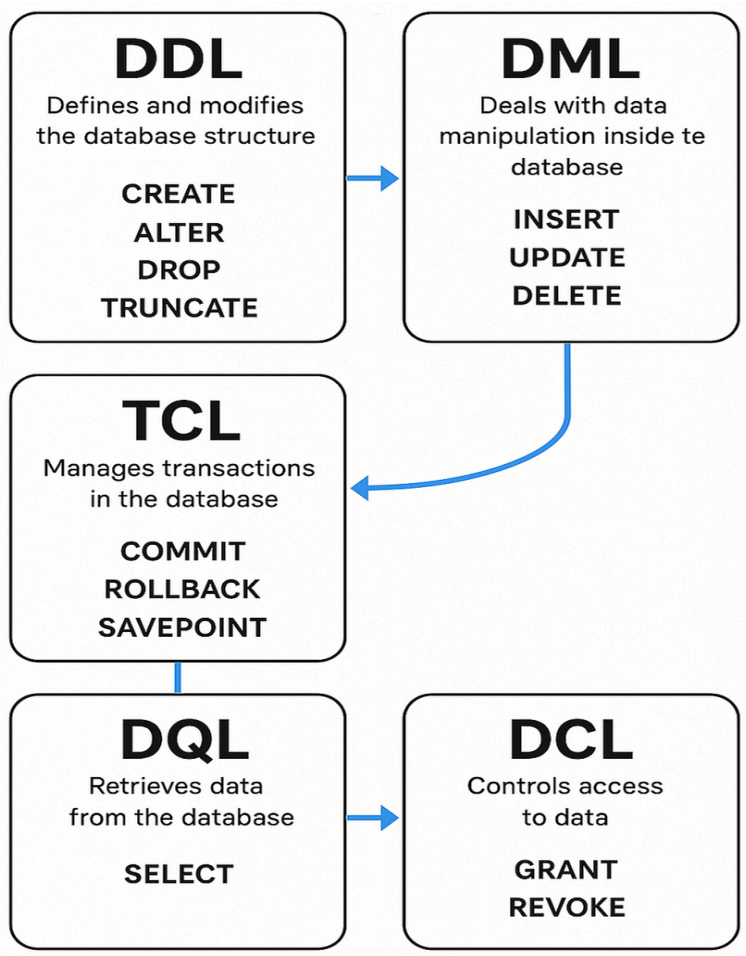
MySQL Commands

DDL → DML - You must define the structure first before you can manipulate data.

DML → TCL - Data manipulation happens within transactions, which TCL controls.

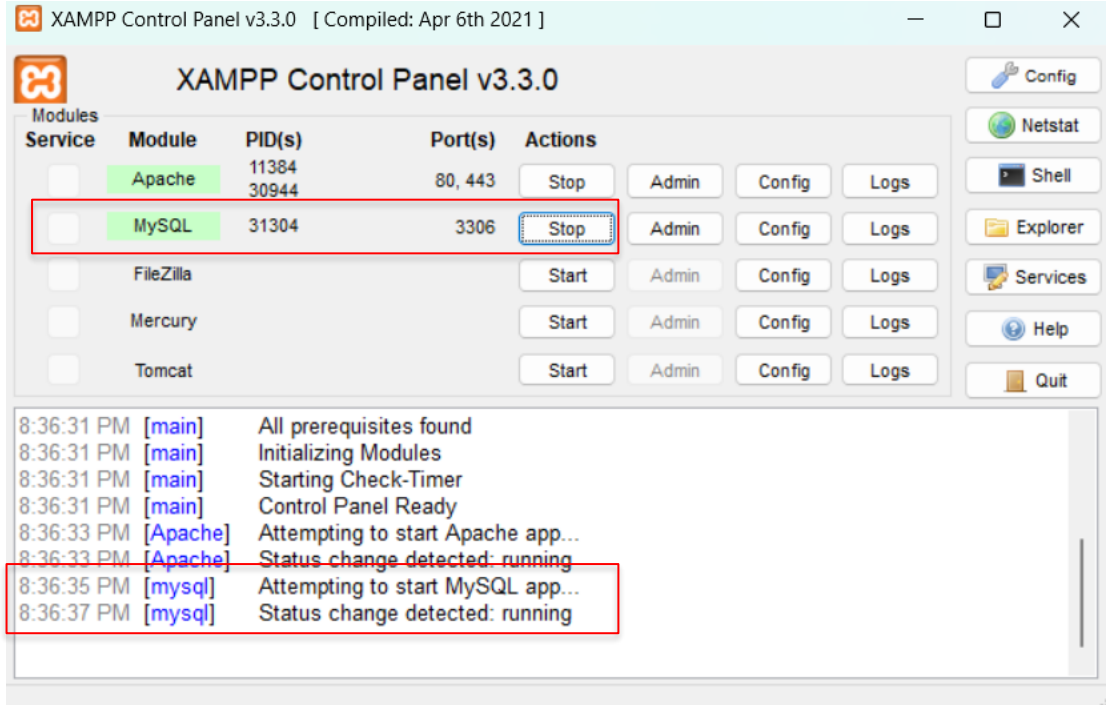
TCL → DQL - Only finalized data states should be queried reliably.

DQL → DCL - Data retrieval depends on access permissions.



Connecting to MySQL with PHP

- **PHP can work with many databases.**
- It can connect to databases using ODBC or connect to them directly.
- PHP also supports SQLite and provides tools that simplify database operations.



Connection to a MySQL Database

- Before you can access data in a database, you must create a connection to the database.
- In PHP, this is done with the **mysqli_connect()** function.

Syntax:

```
mysqli_connect("servername", "username", "password");
```

Parameter	Description
servername	Specifies the server to connect to. Default value is “localhost”
username	Specifies the username to log in with. Default value is “root”
password	Specifies the password to login with. Default value is “ “ (empty)

Example: MySQL Connection

```
<?php
   $con = mysqli_connect("localhost", "root", "");

   if(!$con) {
      die("Could not connect: ". mysqli_error());
   }

   ...
   ...
   mysqli_close($con);
?>
```

Example: Creating a Database

```
<?php
   $con = mysqli_connect("localhost", "root", "");

   if(!$con) {
      die("Could not connect: ". mysqli_error());
   }

   if(mysqli_query("CREATE DATABASE studentDB", $con)) {
     echo "Database created!";
   } else {
     echo "Error creating database: ". mysqli_error();
   }

   mysqli_close($con);
?>
```

mysqli_query() sends a unique query to the currently active database on the server

Making SQL Queries

- PHP function for making queries:
  - `mysqli_query(db_source, query_string);`
- Queries that return information, such as SELECT return a resource
- Other queries return TRUE upon success
- All queries returns FALSE on failure. Best practice is to handle the error → mysqli_error().

Selecting a Database

- Command to be used: **mysqli_select_db (connection, “database name”);**
- Sets the current active database on the server
- Every subsequent call to mysqli_query() will be made on the active database

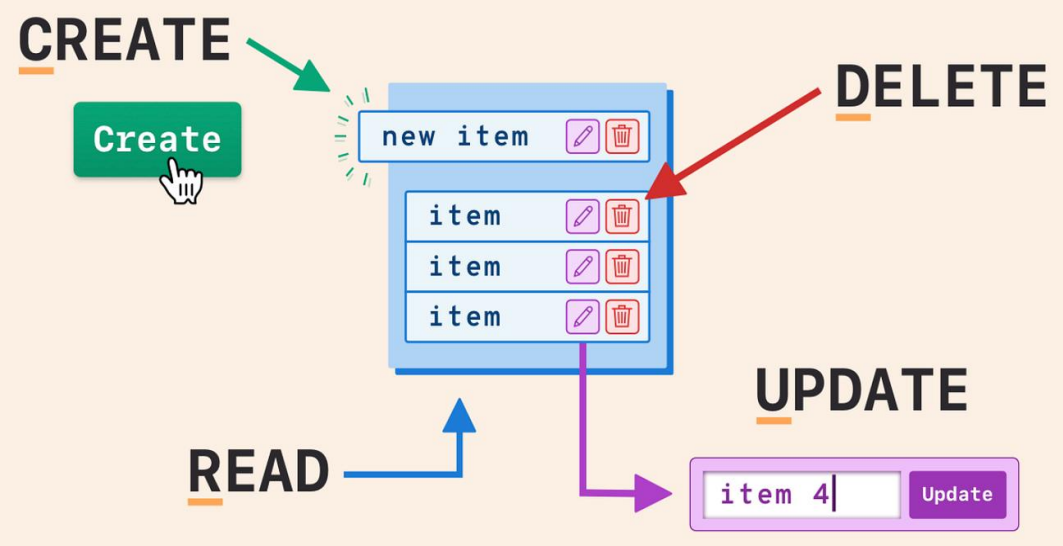
Example:

```
<?php
   $con = mysqli_connect("localhost", "root", "");

   if(!$con) {
      die("Could not connect: ". mysqli_error());
   }

   mysqli_select_db($con, "studentDB");
…
```

Implement CRUD Operations

What Is CRUD?

- **CRUD** represents the four basic database operations:
  - **Create** – Adds new data or records to the database.
  - **Read** – Retrieves or views existing data from the database.
  - **Update** – Modifies existing data in the database.
  - **Delete** – Removes data or records from the database.



CREATE Operation

```
<?php
  $con = mysqli_connect("localhost", "root", "") or
  die("Unable to Connect". mysqli_error());
  mysqli_select_db($con, "studentDB");

  $id = $_POST["id"];
  $name = $_POST["name"];
  $gender = $_POST["gender"];
  $bday = $_POST["Year"].$_POST["Month"].$_POST["Day"];

  $insert = "INSERT INTO tblStudent (id, name, gender, bdate) VALUES ('$id', '$name',
'$gender', '$bday')";

  mysqli_query($conn, $insert);

?>
```

It is assumed that there are data submitted from the PHP/HTML Form

READ Operation

```
<?php
 $con = mysqli_connect("localhost", "root", "")
 or die("Unable to Connect". mysqli_error());
 mysqli_select_db($con,'studentDB');

 if(isset($_POST["id"])) {
 $id = $_POST["id"];
  $query = mysqli_query($con, "SELECT * FROM tblStudent WHERE sid='$id'");
   if($query) {
     while($cell = mysqli_fetch_array($query)) {
       echo "Student ID : <b>". $cell["id"] . "</b><br/>";
       echo "Student Name : <b> " . $cell["name"]. "</b><br/>";
       echo "Student Gender : <b>" . $cell["gender"]. "</b><br/>";
       echo "Student Birthday : <b>" . $cell["bdate"]. "</b><br/>"; }
     }
 }
?>
```

It is assumed that there are data submitted from the PHP/HTML Form

The mysqli_fetch Functions

- **mysqli_fetch_array()** - this is the most flexible function. By default, it returns a single array with both numeric and associative indices (e.g., you can use both $row[0] and $row['username'] for the same value). You can control this behavior by passing a second optional parameter:
  - MYSQLI_NUM makes it behave like mysqli_fetch_row().
  - MYSQLI_ASSOC makes it behave like mysqli_fetch_assoc().
  - MYSQLI_BOTH (the default) returns both types of indices.

```
$query = mysqli_query($con, "SELECT * FROM tblemployee ORDER BY lastname");
  while($retrieve = mysqli_fetch_array($query)){
      echo "ID = ". $retrieve['id']."<br />";
      echo "Last Name = ". $retrieve['lastname']."<br />";
      echo "First Name = ". $retrieve['firstname']."<br />";
      echo "Hire Date = ". $retrieve['hiredate']."<br />";
}
```

The mysqli_fetch Functions

- **mysqli_fetch_assoc()** - This function fetches a result row as an **associative array**. Data is accessed using the column names as keys (e.g., $row['username'], $row['email']), making the code more readable.

```
$query = mysqli_query($con, "SELECT * FROM tblemployee ORDER BY lastname");

while($retrieve = mysqli_fetch_assoc($query)){
  echo "ID = ". $retrieve['id']."<br />";
  echo "Last Name = ". $retrieve['lastname']."<br />";
  echo "First Name = ". $retrieve['firstname']."<br />";
  echo "Hire Date = ". $retrieve['hiredate']."<br />";
}
```

The mysqli_fetch Functions

- **mysqli_fetch_row() -** This function fetches a result row as a strictly **numeric array**. Data can only be accessed using numeric indices starting from 0 (e.g., $row[0], $row[1]).

```
$query = mysqli_query($con, "SELECT * FROM tblemployee ORDER BY lastname");

while($retrieve = mysqli_fetch_row($query)){
  echo "ID = ". $retrieve[0]."<br />";
  echo "Last Name = ". $retrieve[1]."<br />";
  echo "First Name = ". $retrieve[2]."<br />";
  echo "Hire Date = ". $retrieve[3]."<br />";
}
```

The mysqli_fetch Functions

- **mysqli_fetch_object()** - This function fetches a result row as an **object**, with the column names as object properties. Data is accessed using object syntax (e.g., $row->username, $row->email).

```
$query = mysqli_query($DBConnect, "SELECT * FROM tblemployee");
  while($retrieve = mysqli_fetch_object($query)){
      echo ("ID = $retrieve->id <br />");
      echo ("Last Name = $retrieve->lastname <br />");
      echo ("First Name = $retrieve->firstname <br />");
  }
```

UPDATE Operation

```
<?php
  $con = mysqli_connect("localhost", "root", "") or
  die("Unable to Connect". mysqli_error());
  mysqli_select_db($con, "studentDB");

  $id = $_POST["id"];
  $name = $_POST["name"];
  $gender = $_POST["gender"];
  $bday = $_POST["Year"].$_POST["Month"].$_POST["Day"];

  $update = "UPDATE tblStudent SET name='$name', gender='$gender',bdate='$bday' WHERE
id='$id')";

  mysqli_query($con, $update);

?>
```

It is assumed that there are data submitted from the PHP/HTML Form

DELETE Operation

```
<?php
  $con = mysqli_connect("localhost", "root", "") or
  die("Unable to Connect". mysqli_error());
  mysqli_select_db($con, "studentDB");

  $id = $_POST["id"];
  $delete =  "DELETE FROM tblStudent WHERE id='$id';

  mysqli_query($con, $delete);
?>
```

It is assumed that there are data submitted from the PHP/HTML Form

Why CRUD Is Important

- Enables full data management
- Foundation of dynamic web apps
- Used in login systems, dashboards, and forms

PHP–MySQL Application Development

Database Concepts, MySQL Basics, PHP Connections, and CRUD Operations